Orientaatio

Verkko-ohjelmointi

Raine Kauppinen

SeAMK
SEINÄJOEN AMMATTIKORKEAKOULU
SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES

Verkko-ohjelmointi

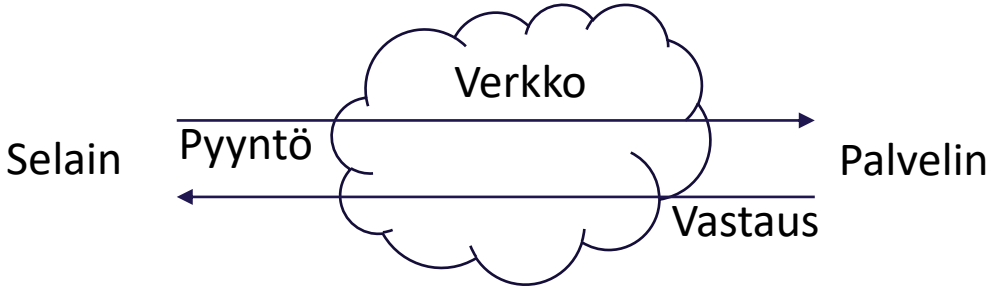
- Verkko-ohjelmoinnilla tarkoitetaan tässä tekniikoita ja menetelmiä, joilla rakennetaan ohjelmia ja järjestelmiä, jotka kommunikoivat verkossa.
- Sisältö perustuu menetelmiin ja tekniikoihin, joita käytetään edellä mainittujen ohjelmien ja järjestelmien toteuttamiseen. Lisäksi käydään läpi mm. rinnakkaisuuden ja säikeiden hyödyntämistä.

Sisältö

- Asiakas-palvelinmalli ja tasoarkkitehtuurit
- Synkronisuus ja asynkronisuus
- HTTP
- AJAX
- REST
- Rinnakkaisuus ja säikeet
- Socket-ohjelmointi ja etämetodikutsut

Asiakas-palvelin ja tasoarkkitehtuuri

- Asiakas – palvelin (pyyntö – vastaus, frontend – backend):



- Kolmitasoarkkitehtuuri (MVC, Model – View – Controller):
  - Model (malli): Tietomalli (tietokanta)
  - View (näkymä): Käyttöliittymä (selain)
  - Controller (ohjain): Logiikka (palvelin)
- 5-tasoarkkitehtuuri, 7-tasoarkkitehtuuri…

Sisältö: Synkronisuus, asynkronisuus ja rinnakkaisuus

- Synkronisuuden ja asynkroniisuuden merkitys
- Synkroniset ja asynkroniset web-tekniikat, esimerkiksi HTTP ja AJAX
- Rinnakkaisuuden merkitys
- Rinnakkaisuuden toteutus säikeiden avulla

Sisältö: Muita tekniikoita ja menetelmiä

- REST-palvelut ja rajapintojen käyttö
- Socket-ohjelmointi ja etämetodikutsut

Verkko-ohjelmoinnin merkitys

- Valtaosa ohjelmistoista ja järjestelmistä sekä esineiden ja teollisen internetin myötä myös laitteista on verkossa.
- Suuri osa tämänhetkisestä ohjelmistotuotannosta hyödyntää siksi verkko-ohjelmoinnin menetelmiä ja tekniikoita (muitakin kuin web-ohjelmoinnin, eli esimerkiksi teollisen internetin hyödyntämiä).

Oppimistavoite

- Tavoitteena on
  - tietää verkko-ohjelmoinnin keskeiset lähtökohdat ja suunnittelu periaatteet kuten asiakas-palvelin, tasoarkkitehtuuri, synkronisuus ja asynkronisuus / rinnakkaisuus,
  - tuntea verkko-ohjelmoinnin eri tilanteisiin soveltuvia tekniikoita kuten HTTP, AJAX, REST, säikeee ja etämetodikutsut, ja
  - osata hyödyntää tekniikoita ohjelmoinnissa.

Asiakas-palvelin -malli ja tasoarkkitehtuuri

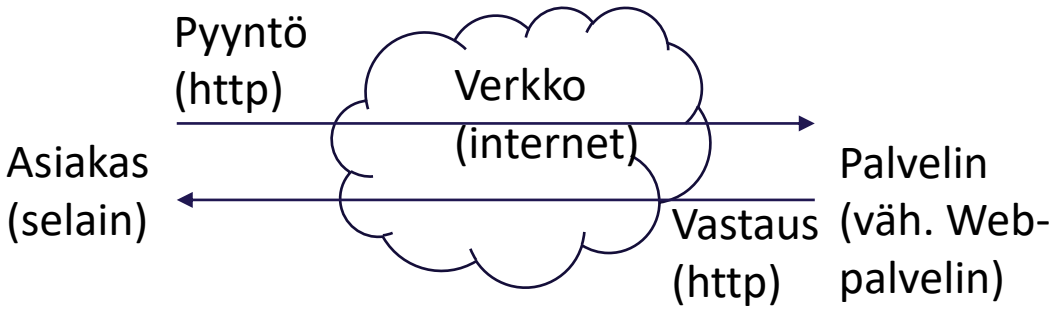
Verkko-ohjelmointi

Raine Kauppinen

SeAMK
SEINÄJOEN AMMATTIKORKEAKOULU
SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES

Kertaus: Asiakas-palvelin –malli ja MVC-tasoarkkitehtuuri (kuten käsitelty Web-ohjelmoinnin perusteissa)

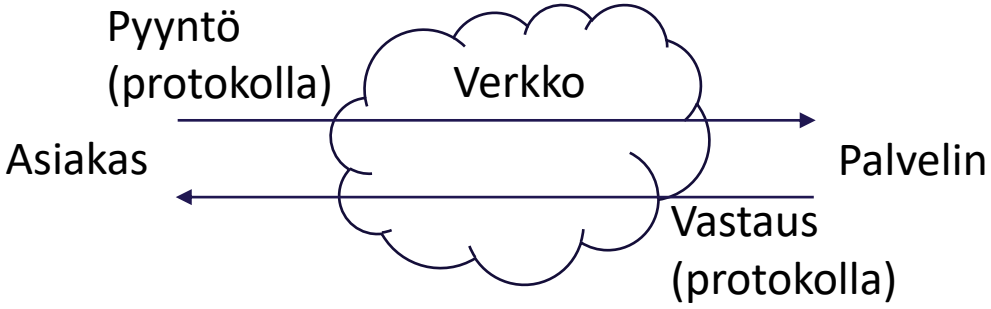
- Asiakas – palvelin (pyyntö – vastaus, frontend – backend):



- Kolmitasoarkkitehtuuri (MVC, Model – View – Controller):
  - Model (malli): Tietomalli (tietokanta)
  - View (näkymä): Käyttöliittymä (selain)
  - Controller (ohjain): Logiikka (palvelin)

Yleinen asiakas-palvelin -malli

- Asiakas-palvelin –malli on muussakin kuin (Web-ohjelmoinnin perusteiden pelkästään) selainpohjaisissa verkkosovelluksissa usein käytetty malli.
- Tämä tarkoittaa mm. seuraavia:
  - Asiakas ei aina ole selain, protokolla http tai palvelin web-palvelin.
  - Myöskään verkko ei aina ole internet tai (laajempien ohjelmistojen tai järjestelmien tapauksessa) niitä voi olla useampia asiakkaan ja palvelimen välissä (esimerkiksi lähiverkko – internet, mahdollisesti väylä- tai solmuteknologioiden kautta).



Yleinen (moni)tasoarkkitehtuuri

- Vastaavasti tasoarkkitehtuuri on muussakin kuin (Web-ohjelmoinnin perusteiden pelkästään) selainpohjaisissa verkkosovelluksissa usein käytetty malli.
- Tämä tarkoittaa mm. seuraavia:
  - Tasoja voi olla muu määrä kuin kolme (kertauskalvolla oleva malli on kolmitasoarkkitehtuuri – yleisesti käytetään termiä monitasoarkkitehtuuri, n-tier architecture).
  - Lisäksi MVC on itse asiassa (arkkitehtuuri)suunnittelumalli, eli tietty versio kolmitasoarkkitehtuurista.
- Kolmitasoarkkitehtuuri:
  - Esitystapa(kerros), presentation tier
  - Logiikka(kerros), logic tier
  - Data(kerros), data tier
- Viisitasoarkkitehtuuri:
  - Esitystapa, presentation.
  - Käyttöliittymä(logiikka), user interface
  - Liiketoiminta(logiikka), business logic
  - Datan käsittely(logiikka), data access
  - Datavarasto, data storage

Synkronisuus ja asynkronisuus

Verkko-ohjelmointi

Raine Kauppinen

SeAMK
SEINÄJOEN AMMATTIKORKEAKOULU
SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES

Synkroninen ja asynkroninen ohjelmointi

- Ohjelmointi on lähtökohtaisesti synkronista (”peräkkäisyys”), mikä tarkoittaa, että yksi asia (esimerkiksi ohjelmointikielen käsky tai lause) suoritetaan kerrallaan ja seuraavan suoritus alkaa vasta, kun edellisen suoritus on päättynyt.
- Asynkronisuus puolestaan tarkoittaa rinnakkaisuutta (”rinnakkaisohjelmointi”, concurrent programming), jossa toisistaan riippumattomia asioita voidaan tehdä yhtä aikaa (eri säikeissä, threads).

Asynkronisyyden hyödyntäminen

- Asynkronisuutta (rinnakkaisuutta) voidaan hyödyntää useimmissa nykyohjelmointikielissä ja -tekniikoissa. Tämä edellyttää kuitenkin aina viestintää, koska asynkronisen osan valmistumista ei jäädä odottamaan, vaan tieto sen valmistumisesta pitää lähettää ja havaita erikseen - tämä vastaanottaa erikseen.
- Osa ohjelmoinnissa käytettävistä välineistä (esimerkiksi kirjastoista ja kehikoista) on asynkronisia, ja asynkronisuutta hyödynnetään esimerkiksi graafisessa käyttöliittymässä, tapahtumankuuntelijoissa (mm. graafiset ja selainkäyttöliittymät) sekä joissakin kirjastofunktoissa tai metodeissa kuten JavaScriptin setTimeout().
- Tyypillisiä esimerkkejä ovat myös asynkroniset funktio/metodi(kutsut)t tai (asiakas-palvelin –mallin) asynkroniset pyynnöt.

Asynkroniset funktiot / metodit

- Asynkronisia funktiota ja metodeita tehdään ohjelmointikielissä tyypillisesti async- ja await-avainsanojen avulla, joko callback-funktioita, tehtäviä (tasks) tai vastaavia hyödyntäen.
  - Esimerkiksi Pythonissa asynkronisella funktiolla on callback-parametri. Funktiota kutsuttaessa ko. parametrin arvoksi annetaan funktio, jota kutsutaan, kun suoritettavan funktion suoritus on valmistunut.
  - Asynkronisesti suoritettavan funktion lopussa on callback-kutsu (vrt. return), jonka mukana välitetään tarvittaessa paluuarvo.
- Asynkronisiin funktioihin palataan rinnakkaisohjelmoinnin ja säikeiden käsittelyn yhteydessä tarkemmin.

Asynkroniset pyynnöt - HTTP

- Web-ohjelmoinnissa asynkronisuutta hyödynnetään usein tekemällä asynkronisia pyyntöjä palvelimelle, jolloin sivua ei tarvitse ladata uudestaan ja odottaa vastausta, vaan voidaan pysyä samalla sivulla ja odottaa sekä käsitellä vastaus taustalla. Tätä käytetään esimerkiksi hakemaan tarvittavia tietoja taustalla tiedostosta tai tietokannasta ja päivittämään ne sivulle, kun ne saapuvat.
  - Esimerkiksi JavaScriptissä alkuperäinen toteutustapa tähän on AJAX (Asynchronous JavaScript and XML).
  - Nykyselaimet tukevat myös uudempaa Fetch APIa, joka tekee saman.

AJAX ja Fetch API

Verkko-ohjelmointi

Raine Kauppinen

SeAMK
SEINÄJOEN AMMATTIKORKEAKOULU
SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES

AJAX (Asynchronous JavaScript and XML)

- AJAX on verkkosivujen asynkronisen päivittämisen mahdollistava yhdistelmä, jossa hyödynnetään
  - selaimissa käytössä olevaa XMLHttpRequest-objektia, jolla saadaan pyydettyä dataa (tiedosto) palvelimelta, ja
  - JavaScriptiä sekä HTML:n DOMia (document object model), joilla näytetään ja käsitellään dataa.
- XML-osa nimessä on historiallinen ja nykyään harhaanjohtava: XML:n lisäksi, ja nykyään sijasta, data on tyypillisesti esimerkiksi JSON-muotoista tai pelkkää tekstiä.

AJAXin hyödyntäminen

- AJAXilla toteutetaan tyypillisesti seuraavia:
  - Datan lukeminen palvelimelta sen jälkeen, kun sivu on ladattu.
  - Sivun päivitys tapahtumaohjatusti lataamatta sivua uudelleen.
  - Datan lähettäminen palvelimelle taustalla.

AJAXin käyttö

- AJAXin käytön ydin on XMLHttpRequest-olio, jonka avulla asynkroninen pyyntö toteutetaan seuraavasti:
  - Luodaan XMLHttpRequest-objektille.
  - Määritellään callback-funktio.
  - Määritellään XMLHttpRequest-objektille pyynnön tyyppi (get tai post) sekä ladattava tiedosto.
  - Lähetetään pyyntö.

Fetch API

- XMLHttpRequest-objektin käyttö on (syntaksiltaan) melko työlästä.
- Uudempi vaihtoehto, joka on yksinkertaisempi, on Fetch API eli rajapinta. Se perustuu asyncin ja awaitin käyttöön.

HTTP

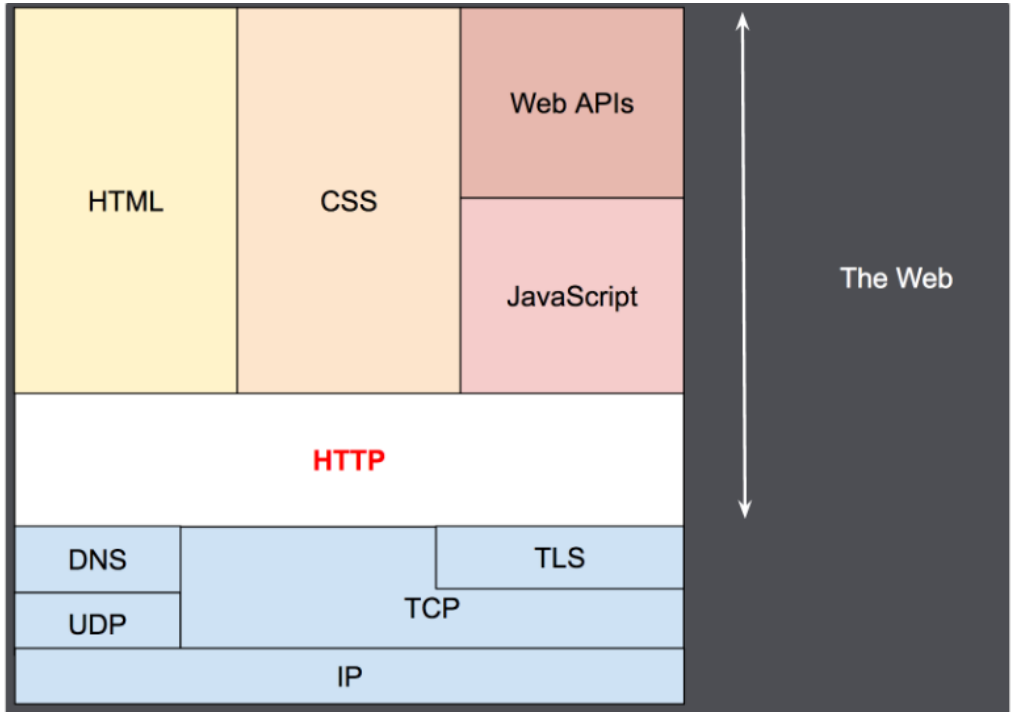
Verkko-ohjelmointi

Raine Kauppinen

SeAMK
SEINÄJOEN AMMATTIKORKEAKOULU
SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES

HTTP (HyperText Transfer Protocol)

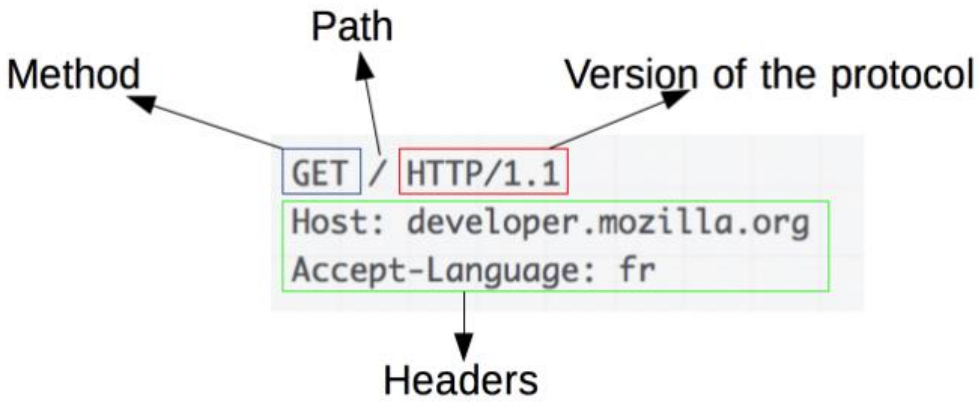
- Web-sovellukset rakentuvat HTTP:n päälle, eli niiden asiakas-palvelin –mallin mukainen viestintä, pyynnöt ja vastaukset ovat HTTP:tä.
- HTTP suunniteltiin alkujaan ihmisen(kin) luettavaksi (HTTP/1.1 ja aiemmat), joskin nykyään (HTTP/2) viestit ovat binaarikehyksen (frame) sisällä.



(Kuva: developer.mozilla.org)

HTTP-pyyntö (request)

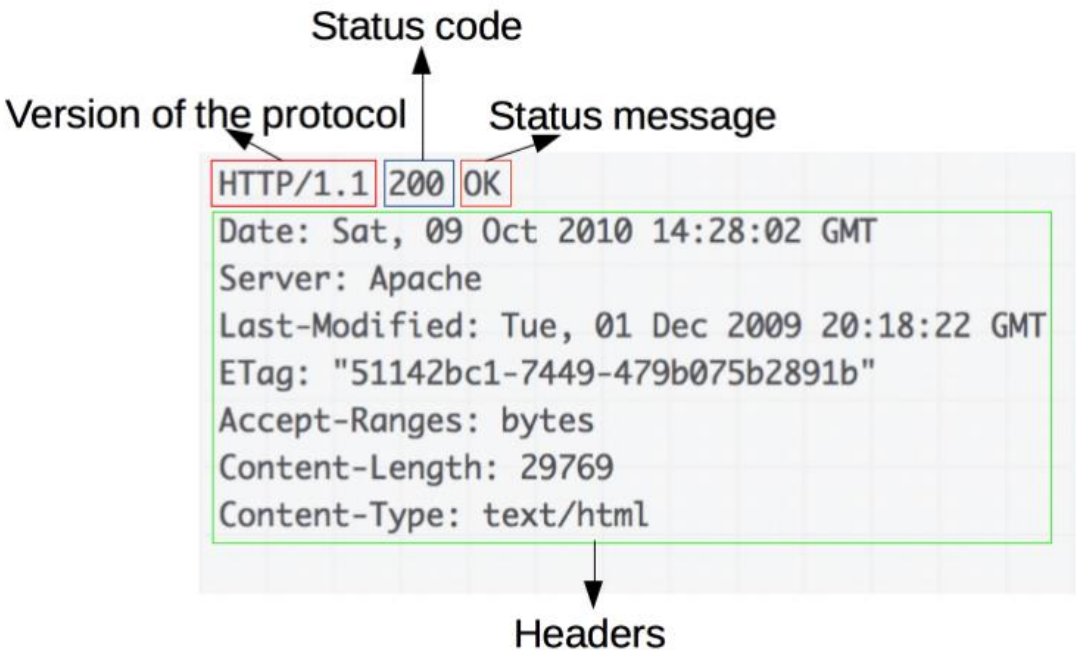
- HTTP-pyynnössä on seuraavat osat:
  - Method: Operaatio, jonka asiakas haluaa tehdä (tyypillisesti GET tai POST, mutta voi olla myös esimerkiksi OPTIONS tai HEAD).
  - Path: Polku noudettavaan resurssiin (riisuttuna esimerkiksi protokollasta, domainista ja portista, jos ilmeisiä).
  - Käytettävä protokollan versio.
  - Headers: Valinnaiset otsakkeet, joilla välitetään lisätietoa.
  - Body: Joidenkin operaatioiden, mm. POST yhteydessä pyynnössä on myös runko (body) kuten vastauksissa.



(Kuva: developer.mozilla.org)

HTTP-vastaus (reply)

- HTTP-vastauksessa on seuraavat osat:
  - Käytettävä protokollan versio.
  - Status code: Tilakoodi, joka kertoo onnistuiko pyynnön suorittaminen.
  - Status message: Lyhyt tilakoodin merkityksen kuvaava teksti.
  - Headers: Valinnaiset otsakkeet, joilla välitetään lisätietoa.
  - Body: Valinnainen osa, joka sisältää haetun resurssin.



(Kuva: developer.mozilla.org)

HTTP:n GET ja POST

GET – datan pyyntö

- Pyynnöt voidaan tallettaa välimuistiin (cache).
- Pyynnöt voidaan säilyttää selainhistoriassa ja niistä voi tehdä kirjanmerkin.
- Pyynnön data (esimerkiksi lomakkeen kenttien nimet ja arvot) välitetään pyynnön URL:ssa ja se näkyy selaimen osoiterivillä. Datan määrä on rajoitettu (URL:n maksimipituus on 2048 merkkiä) ja siinä voi olla vain ASCII-merkkejä.
- Käytetään datan pyytämiseen resurssilta, ei muokkaamiseen.

POST – datan lähetys

- Pyyntöjä ei talleteta välimuistiin.
- Pyynnöt eivät jää selainhistoriaan, eikä niistä voi tehdä kirjanmerkkiä.
- Pyynnön data välitetään rungossa (body). Datan määrää ei ole rajoitettu ja se voi sisältää myös binaaridataa.
- Käytetään datan lähettämiseen palvelimelle resurssin luomiseksi tai muokkaamiseksi.

HTTP:n APIt

- HTTP:n yleisin ohjelmointirajapinta (API, Application Programming Interface) on XMLHttpRequest API.
- Vastaavat ominaisuudet toteuttaa uudempi FetchAPI, joka on tehokkaampi ja joustavampi.

APIt, web servicet ja REST

Verkko-ohjelmointi

Raine Kauppinen

SeAMK
SEINÄJOEN AMMATTIKORKEAKOULU
SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES

API (ohjelmointirajapinta)

- APIlla (Application Programming Interface) tarkoitetaan yleisesti ohjelmointirajapintoja.
- Ohjelmointirajapinnat voivat toimia esimerkiksi ohjelmistojen tai ohjelmiston ja laitteiston välillä.
- Ohjelmointirajapinnan keskeinen osa on sen dokumentointi, joka mahdollistaa sujuvan käytön.

Web service (www-sovelluspalvelu)

- Web service on ”ohjelmistojärjestelmä, joka mahdollistaa keskenään yhteensopivien tietokoneiden välisen vuorovaikutuksen tietoverkon yli” (W3C).
- Käytännössä web servicet ovat soveltuvilla teknologioilla toteutettuja web-pohjaisia ohjelmointirajapintoja, joiden avulla palvelin tarjoaa muilla tietokoneilla toimiville ohjelmistoille jonkin palvelun tiettyä protokollaa, esimerkiksi HTTP:tä hyödyntäen.

Web servicen toteutus: SOAP, WSDL ja UDDI

- Web service voidaan toteuttaa erityisesti sitä varten määritellyillä teknologioilla (jotka ovat useimmiten XML-pohjaisia). Tyypilliset tällaiset ovat seuraavat:
  - SOAP (Simple Object Access Protocol), kieli ja protokolla komennon ja vastauksen käsittelyyn,
  - WSDL (Web Service Description Language), jonka avulla voidaan määritellä SOAP-protokollalla toteutetun palvelun rajapinta eli käytettävien pyyntöjen ja vastausten muoto, ja
  - UDDI (Universal Description Discovery and Integration), standardoitu tapa julkaista ja etsiä eri palveluiden kuvaus- eli metadataa.

Web servicen toteutus: REST

- REST (REpresentational State Transfer) on arkkitehtuurimalli, joka on vaihtoehtoinen tapa toteuttaa web service käyttämällä vain HTTP-protokollaa.
- REST ei siis ole protokolla tai standardi, vaan (arkkitehtuuri)malli, joka määrittelee suunnittelua ja toteutusta ohjaavia periaatteita.
  - Näitä periaatteita noudattavista palvelujen rajapinnoista käytetään usein nimitystä ”RESTful”.
  - Web service (tai muu verkossa tarjolla oleva ohjelmointirajapinta), joka on RESTin mukainen on REST API.

RESTin periaatteet

- Periaatteita on kuusi, joista viimeinen on valinnainen:
  1. Yhtenäinen rajapinta (uniform interface).
  2. Asiakas-palvelin –malli (client-server).
  3. Tilattomuus (stateless).
  4. Välimuistin käyttö (cacheable).
  5. Kerrosarkkitehtuuri (layered system).
  6. Valinnainen: Ladattava ohjelmakoodi (code on demand)

REST-pohjaisen web servicen / APIn käyttö

- Asiakaspäässä REST-pohjaisen web servicen tai APIn käytön voi ajatella olevan verkossa olevan datalähteen hyödyntämistä.
  - Datalähteellä eli palvelulla on tietty osoite ja sitä käytetään palvelun määrittelemällä tavalla tarvittavia HTTP-pyyntöjä lähettämällä (ja vastauksia käsittelemällä).
- Asiakaspää voi olla toteutettu millä tahansa kielellä ja teknologialla, joka mahdollistaa HTTP-pyyntöjen luomisen ja HTTP-vastausten käsittelyn.

REST ja CRUD

- RESTin mukainen palvelu voi mahdollistaa käsiteltävälle resurssille erilaisia operaatioita. Nämä ovat usein CRUD-periaatteen (Create, Read, Update, Delete) mukaisia ja niille määritellään REST APIssa tyypillisesti vastaavat HTTP-operaatiot, esimerkiksi:
  - POST: Create
  - GET: Read
  - PUT: Update
  - DELETE: Delete
- Tämä toteuttaa RESTin yhdenmukaisen rajapinnan periaatteen.

Endpoint

- Endpoint on RESTin mukaisessa palvelussa keskeinen käsite. Se on yleensä tietty osoite, jota (tietyllä HTTP-operaatiolla käyttämällä) palvelua käytetään. Endpointin nimi (osoite) kuvaa yleensä sen tarkoituksen.
- Tyypillinen tapa dokumentoida RESTin mukainen palvelu(rajapinta) on luetella sen endpointit lyhyen kuvaustekstin kanssa.

API-avain (API key)

- RESTin mukaisen julkiset palvelut voivat olla täysin avoimia, jolloin niitä voi käyttää omissa toteutuksissa suoraan.
- Osa palveluista puolestaan vaatii tunnistautumisen, joka toteutuu usein API-avaimen (yksilöivä tunniste) avulla. API-avaimen saa tyypillisesti rekisteröitymällä.
- Lisäksi osa palveluista on maksullisia.

Julkisten web-APIen kokoelmia

- Esimerkiksi seuraavilla sivustoilla on avoimien web-APIen tietoja (, joista kaikki eivät ole RESTin mukaisia):
  - Täysin avoimia: https://apipheny.io/free-api/
  - API-avaimen vaativia (ja tyypillisesti tietyn pyyntömäärän jälkeen maksullisia): https://rapidapi.com/collection/list-of-free-apis

Rinnakkaisuus ja säikeet

Verkko-ohjelmointi

Raine Kauppinen

SeAMK
SEINÄJOEN AMMATTIKORKEAKOULU
SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES

Rinnakkaisuus ja säikeet

- Aiemmin käsiteltiin asynkronisia funktioita, joiden suorittamista ei jääty odottamaan, vaan suoritusta jatkettiin ja rinnakkaisen funktion suorituksen valmistuessa siirryttiin käsittelemään sen tulosta (callback).
- Tässä käsitellään säikeitä, joiden avulla (toisistaan riippumattomat) tehtävät voidaan rinnakkaistaa yksittäisessä prosessissa (rinnakkaisia prosesseja ei käsitellä), jonka jatkaminen tai valmistuminen edellyttää kaikkien tehtävien valmistumista.

Säikeiden hyödyntäminen

- Säikeitä voidaan hyödyntää useimmissa nykyohjelmointikielissä ja -tekniikoissa. Tämä edellyttää tapaa (mm.) määritellä, käynnistää ja yhdistää säikeet sekä määritellä kullekin säikeelle tehtävänsä.
- Tyypillinen käyttö säikeille ovat I/O-operaatiot (luku- ja kirjoitus-), joka ovat hitaita eli sisältävät prosessorin kannalta suurimmaksi osaksi odottamista.
- Tyypillisiä esimerkkejä säikeiden käyttöön ovat niputetut levy-, tietokanta- tai ulkoisten resurssien operaatiot.

Säikeiden toteuttaminen

- Säikeitä varten on tyypillisesti oma kirjastonsa, joka tarjoaa rajapinnan eli funktiot tai metodit säikeiden käsittelyyn.
  - Esimerkiksi Pythonissa on moduli threading, jossa on säiettä edustava luokka Thread.
  - Säikeen voi määritellä ko. luokan konstruktorilla, käynnistää luokan metodilla start ja yhdistää metodilla join.

Socket-ohjelmointi

Verkko-ohjelmointi

Raine Kauppinen

SeAMK
SEINÄJOEN AMMATTIKORKEAKOULU
SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES

Pistokeohjelmointi (socket programming)

- Pistokkeilla (sockets) tarkoitetaan prosessien väliseen tai verkon yli tapahtuvaan tiedonsiirtoon käytettäviä ohjelmakohtaisia (prosessikohtaisia) osoitteita, jotka ohjelmassa voidaan luoda tiedonsiirtoa varten.
- Pistokkeet mahdollistavat mm. pysyvät kaksisuuntaiset yhteydet ohjelmien (prosessien) välillä.

Pistokkeiden hyödyntäminen

- Pistokkeita voidaan hyödyntää myös paikallisten ohjelmien (prosessien) välillä, mutta tässä keskitytään verkon yli tapahtuvaan pistokkeiden hyödyntämiseen.
  - Network socket tai Internet socket viittaa usemmiten IP-protokollan käyttöön (TCP/IP), jolloin socket-osoite on protokollan, ip-osoitteen ja porttinumeron yhdistelmä.
  - Palvelinohjelmat luovat pistokkeen tai pistokkeet joita ne kuuntelevat ja joihin asiakasohjelmistot ottavat yhteyden. Tällöin muodostuu socket-pari.
  - Esimerkiksi Windowsissa muodostetut parit näkee komentokehotteessa `netstat`-komennolla.

WebSocket

- WebSocket on HTML5:n yhteydessä kehitetty internet-protokolla (ws ja wss, vrt. http ja https), joka mahdollistaa kaksisuuntainen viestinnän. WebSockets on API ko. protokollan käyttöön.
  - Http-yhteys voidaan muuttaa (upgrade) WebSocket-yhteydeksi.
  - Tällöin palvelin voi mm. päivittää tietoa asiakkaalle (esimerkiksi web-sivun sisältöä) ilman, että asiakkaan tarvitsee lähettää uutta pyyntöä (refresh).
- Koska WebSocketsin (täysi) toteuttaminen ja selaintuen laajeneminen vei aikaa, sitä vastaavan tai laajennetun toiminnallisuuden tarjoavia erillisiä kirjastoja kuten Socket.IO on myös olemassa ja edelleen laajalti käytössä.
  - Socket.IO käyttää WebSocketteja, jos ne ovat käytettävissä ja muutoin muita tekniikoita, joilla saadaan aikaa WebSocketteja vastaava yhteys HTTP:n yli. Socket.IO:ssa on myös joitakin laajennoksia kuten automaattinen yhteyden palautus.

Etämetodikutsut

Verkko-ohjelmointi

Raine Kauppinen

SeAMK
SEINÄJOEN AMMATTIKORKEAKOULU
SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES

Etämetodikutsut (RPC, remote procedure call)

- Etämetodikutsulla (etäproseduurikutsulla) tarkoitetaan tilannetta, jossa ohjelma kutsuu funktiota tai metodia, joka suoritetaan toisessa osoiteavaruudessa (tyypillisesti toisessa verkon tietokoneessa). Kyseessä on siis eräs hajautetun ohjelmoinnin tai laskennan muoto (distributed computing).
- Kutsu ohjelmoidaan niin kuin se olisi paikallinen, eli siinä ei ole etäyhteyden yksityiskohtia.
- Kyseessä on myös eräs asiakas-palvelin –mallin ilmentymä, sillä kutsuja on asiakas ja etämetodin suorittaja palvelin. Viestintä myös perustuu useimmiten pyyntö-vastaus –pariin.

Etämetodikutsujen hyödyntäminen

- Muutamia esimerkkejä etämetodikutsujen hyödyntämisestä ovat seuraavat:
  - NFS (network file system) – yksi merkittävimmistä.
  - SAPin remote function call – SAP-järjestelmä voi kutsua eli käynnistää funktion suorituksen toisessa SAPissa.
  - Googlen Web Toolkit – käyttää etämetodikutsuja (asynkronisia) palvelimen palvelujen kutsumiseen.

RPC:tä tukevia ohjelmointikieliä ja tekniikoita

- Esimerkiksi seuraavat ohjelmointikielissä ja tekniikoissa on tuki etämetodikutsuille:
  - Java – Java RMI (Java Remote Method Invocation).
  - Python – RPyC (Remote Python Calls)
  - SOAP (XML-RPC:n seuraaja) – Etämetodikutsut Web Serviceissä.
  - JSON-RPC – JSON-muotoiset etämetodikutsut.
  - WCF (ennen Indigo) – Windows Communication Foundation, .NET-etämetodikutsut

RPC vs. REST(full) yms.

RPC:tä voi käyttää, kun palvelimelta halutaan kutsua toimintoa tai raskasta laskentaa tms.

- Esimerkkejä:
  - Kuvan ottaminen etänä laitteen kameralla
  - Koneoppimisalgoritmin suorituksen käynnistys palvelimella
  - Rahansiirto tililtä toiselle hajautetussa pankkijärjestelmässä
  - Palvelimen uudelleenkäynnistys etänä
- Käytännössä nykyään lähinnä erityisissä tilanteissa, mm. hajautetussa rinnakkaislaskennassa

REST:iä voi käyttää, kun halutaan tehdä CRUD-operaatioita palvelimella olevaan dataan

- Esimerkkejä:
  - Tuotteen lisääminen tietokantaan (C)
  - Soittolistan kappaleiden nimien haku (R)
  - Henkilön osoitteen päivitys (U)
  - Blogikirjoituksen poisto (D)
- Käytännössä nykyään yleisempi, ts. saatetaan käyttää myös tilanteissa, jotka voisi toteuttaa myös RPC:llä